The first is the use of magic numbers. The values 0.9 and 0.8 represent discount rates, but they carry no meaning on their own and rely entirely on the inline comments to explain what they do. Clausen (2021) describes this as a common but easily avoidable smell, since any future change to a discount rate means hunting through the function body and hoping every occurrence gets updated consistently, rather than changing a single named value in one place.

The second smell is the long conditional chain handling multiple discount rules within a single method. Right now the function is only responsible for three cases, but every new item type or pricing rule would mean extending the same if-elif-else block, which makes the function progressively harder to read and increases the risk of accidentally breaking existing logic when adding new conditions. Alls (2023) refers to this pattern as a violation of the single responsibility principle, since the function is effectively handling both the iteration over items and the discount decision logic for every item type at once.

For a simpler fix, the magic numbers can be replaced with named constants such as BOOK_DISCOUNT = 0.9 and ELECTRONICS_DISCOUNT = 0.8, which immediately makes the intent clearer without changing the structure of the function itself.

For a more scalable fix, the conditional logic can be replaced using the Strategy pattern, where each discount type is encapsulated in its own class implementing a shared interface. Eland and Smith (2023) discuss this exact substitution, replacing conditional branching with polymorphism, as one of the more effective refactoring techniques for reducing long term maintenance cost, since new discount types can then be added as new classes rather than by editing existing logic.

Below is a refactored version of the code using constants. This removes the magic numbers and makes the discount values easy to find and update in one place, but the conditional chain itself is still there, so adding a new item type still means editing this function directly.

```
BOOK_DISCOUNT = 0.9

ELECTRONICS_DISCOUNT = 0.8



def calculate_total_price(items):

    total = 0

    for item in items:

        if item['type'] == 'book':
```

```
            total += item['price'] * BOOK_DISCOUNT
        elif item['type'] == 'electronics':
            total += item['price'] * ELECTRONICS_DISCOUNT
        else:
            total += item['price']
    return total
```

References

Alls, J. (2023) *Clean code with C#: refactor your legacy C# code base and improve application performance using best practices*. 2nd edn. Birmingham: Packt Publishing.

Clausen, C. (2021) *Five lines of code*. Shelter Island, NY: Manning Publications.

Eland, M. and Smith, S. (2023) *Refactoring with C#: safely improve .NET applications and pay down technical debt with Visual Studio, .NET 8, and C# 12*. 1st edn. Birmingham: Packt Publishing.